



Distributed Systems: Concepts and Design

Edition 5

**By George Coulouris, Jean Dollimore, Tim Kindberg and Gordon Blair
Addison-Wesley ©Pearson Education 2012**

Chapter 2 Exercise Solutions

- 2.1 Provide three specific and contrasting examples of the increasing levels of heterogeneity experienced in contemporary distributed systems as defined in Section 2.2.

2.1 Ans.

Heterogeneity exists in many areas of a contemporary distributed system including in the areas of hardware, operating systems, networks and programming languages. We look at the first three as examples:

- In terms of hardware, distributed systems are increasingly heterogeneous featuring (typically Intel-based) PCs, smart phones, resource-limited sensor nodes, and resource-rich cluster computers or multi-core processors.
- In terms of operating systems, a distributed system may include computers running Windows, MAC OS, various flavours of Unix, and also more specialist operating systems for smart phones or sensor nodes.
- In terms of networks, the Internet is also increasingly heterogeneous embracing wireless technologies and ad hoc styles of networking.

-
- 2.2 What problems do you foresee in the direct coupling between communicating entities that is implicit in remote invocation approaches? Consequently, what advantages do you anticipate from a level of decoupling as offered by space and time uncoupling? Note: you might want to revisit this answer after reading Chapters 5 and 6.

2.2 Ans.

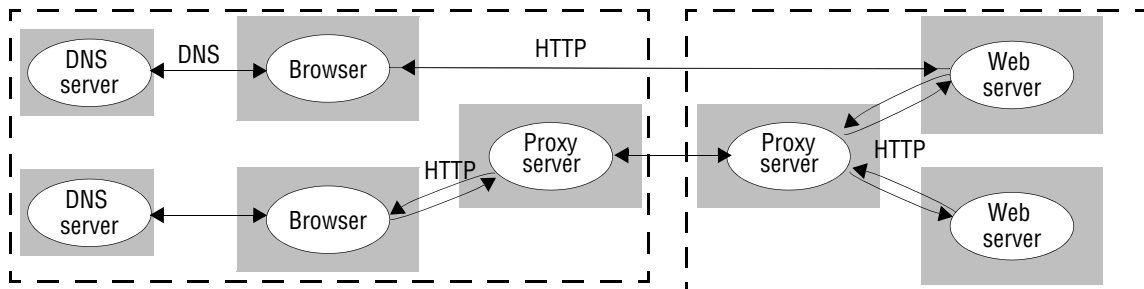
The client is intrinsically bound to the server and vice versa and this is inflexible in terms of dealing with failure, for example if the server fails and a backup server takes over managing requests. More generally, this level of coupling makes it hard to deal with change.

Clients and servers must exist at the same time and hence it is not possible to operate in more volatile environments when either party may be unavailable, for example disconnected in the case of a mobile node.

The benefits of space uncoupling is in providing more degrees of freedom in dealing with change, for example if a new server starts dealing with requests.

The benefit of time uncoupling is in allowing entities to communicate when entities may come and go.

-
- 2.3 Describe and illustrate the client-server architecture of one or more major Internet applications (for example the Web, email or netnews).

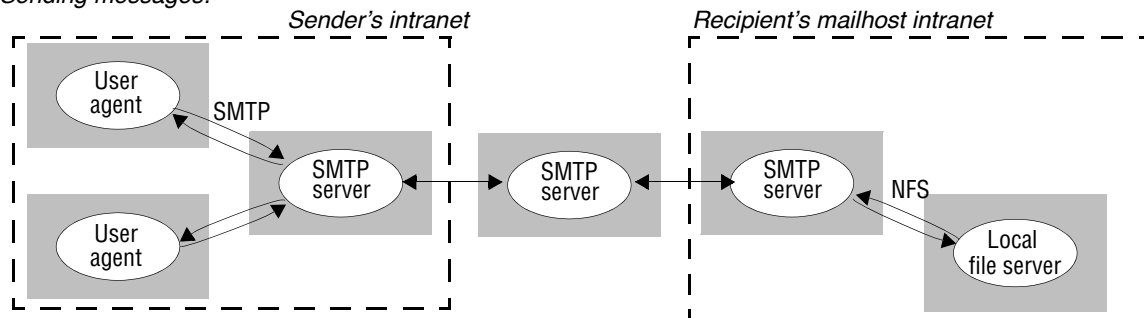
Web:

Browsers are clients of Domain Name Servers (DNS) and web servers (HTTP). Some intranets are configured to interpose a Proxy server. Proxy servers fulfil several purposes – when they are located at the same site as the client, they reduce network delays and network traffic. When they are at the same site as the server, they form a security checkpoint (see pp. 107 and 271) and they can reduce load on the server.

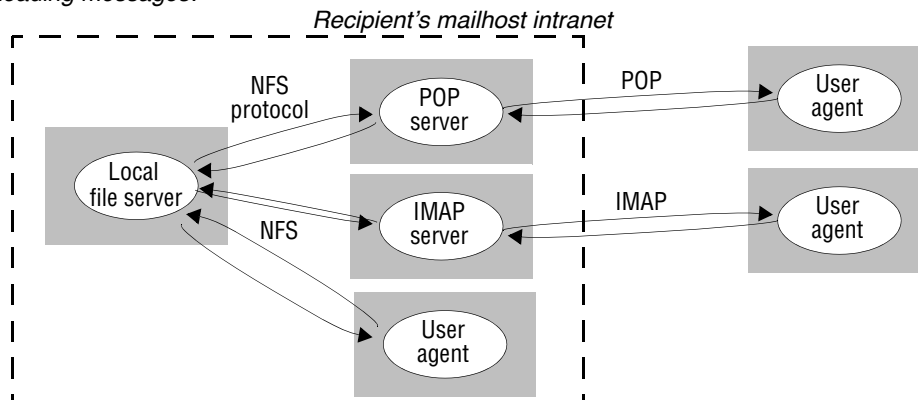
N.B. DNS servers are also involved in all of the application architectures described below, but they are omitted from the discussion for clarity.

Email:

Sending messages:



Reading messages:



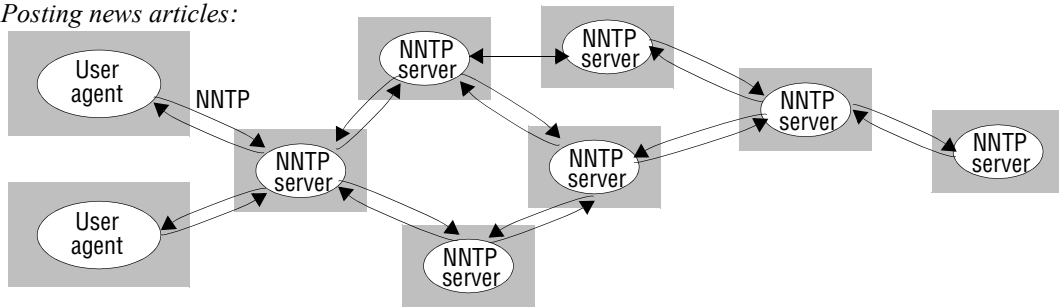
Sending messages: User Agent (the user's mail composing program) is a client of a local SMTP server and passes each outgoing message to the SMTP server for delivery. The local SMTP server uses mail routing tables to determine a route for each message and then forwards the message to the next SMTP server on the chosen route. Each SMTP server similarly processes and forwards each incoming message unless the domain name in the message address matches the local domain. In the latter case, it attempts to deliver the message to local recipient by storing it in a mailbox file on a local disk or file server.

Reading messages: User Agent (the user's mail reading program) is *either* a client of the local file server or a client of a mail delivery server such as a POP or IMAP server. In the former case, the User Agent reads messages directly from the mailbox file in which they were placed during the message delivery. (Examples of such user agents are the UNIX *mail* and *pine* commands.) In the latter case, the User Agent requests information about the contents of the user's mailbox file from a POP or IMAP server and receives messages

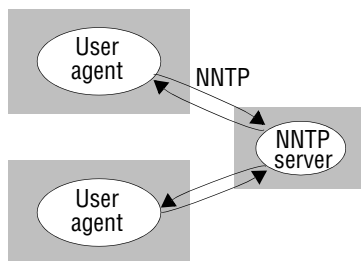
from those servers for presentation to the user. POP and IMAP are protocols specifically designed to support mail access over wide areas and slow network connections, so a user can continue to access her home mailbox while travelling.

Netnews:

Posting news articles:



Browsing/reading articles:



Posting news articles: User Agent (the user's news composing program) is a client of a local NNTP server and passes each outgoing article to the NNTP server for delivery. Each article is assigned a unique identifier. Each NNTP server holds a list of other NNTP servers for which it is a newsfeed – they are registered to receive articles from it. It periodically contacts each of the registered servers, delivers any new articles to them and requests any that they have which it has not (using the articles' unique id's to determine which they are). To ensure delivery of every article to every Netnews destination, there must be a path of newsfeed connections from that reaches every NNTP server.

Browsing/reading articles: User Agent (the user's news reading program) is a client of a local NNTP server. The User Agent requests updates for all of the newsgroups to which the user subscribes and presents them to the user.

-
- 2.4 For the applications discussed in Exercise 2.1 what placement strategies are employed in implementing the associated services?.

2.4 Ans.

We first consider the mapping to multiple machines (covering partitioning and replication strategies):

Web: Web page masters are held in a file system at a single server. The information on the web as a whole is therefore partitioned amongst many web servers.

Replication is not a part of the web protocols, but a heavily-used web site may provide several servers with identical copies of the relevant file system using one of the well-known means for replicating slowly-changing data (Chapter 15). HTTP requests can be multiplexed amongst the identical servers using the (fairly basic) DNS load sharing mechanism described on page 169. In addition, web proxy servers support replication through the use of cached replicas of recently-used pages and browsers support replication by maintaining a local cache of recently accessed pages.

Mail: Messages are stored only at their destinations. That is, the mail service is based mainly on partitioning, although a message to multiple recipients is replicated at several destinations.

Netnews: Each group is replicated only at sites requiring it.

In terms of caching (and proxies), web servers cooperate with Proxy servers to minimize network traffic and latency. Responsibility for consistency is taken by the proxy servers - they check the modification dates of pages frequently with the originating web server.

The web also features significant use of mobile code, for example through applets where code is downloaded and run on the browser machine.

- 2.5 A search engine is a web server that responds to client requests to search in its stored indexes and (concurrently) runs several web crawler tasks to build and update the indexes. What are the requirements for synchronization between these concurrent activities?

2.5 Ans.

The crawler tasks could build partial indexes to new pages incrementally, then merge them with the active index (including deleting invalid references). This merging operation could be done on an off-line copy. Finally, the environment for processing client requests is changed to access the new index. The latter might need some concurrency control, but in principle it is just a change to one reference to the index which should be atomic.

- 2.6 The host computers used in peer-to-peer systems are often simply desktop computers in users' offices or homes. What are the implications of this for the availability and security of any shared data objects that they hold and to what extent can any weaknesses be overcome through the use of replication?

2.6 Ans.

Problems:

- people often turn their desktop computers off when not using them. Even if on most of the time, they will be off when user is away for an extended time or the computer is being moved.
- the owners of participating computers are unlikely to be known to other participants, so their trustworthiness is unknown. With current hardware and operating systems the owner of a computer has total control over the data on it and may change it or delete it at will.
- network connections to the peer computers are exposed to attack (including denial of service).

The importance of these problems depends on the application. For the music downloading that was the original driving force for peer-to-peer it isn't very important. Users can wait until the relevant host is running to access a particular piece of music. There is little motivation for users to tamper with the music. But for more conventional applications such as file storage availability and integrity are all-important.

. Solutions:

Replication:

- if data replicas are sufficiently widespread and numerous, the probability that all are unavailable simultaneously can be reduced to a negligible level.
- one method for ensuring the integrity of data objects stored at multiple hosts (against tampering or accidental error) is to perform an algorithm to establish a consensus about the value of the data (e.g. by exchanging hashes of the object's value and comparing them). This is discussed in Chapter 15. But there is a simpler solution for objects whose value doesn't change (e.g. media files such as music, photographs, radio broadcasts or films).

Secure hash identifiers:

- The object's identifier is derived from its hash code. The identifier is used to address the object. When the object is received by a client, the hash code can be checked for correspondence with the identifier. The hash algorithms used must obey the properties required of a secure hash algorithm as described in Chapter 7.
-

- 2.7 List the types of local resource that are vulnerable to an attack by an untrusted program that is downloaded from a remote site and run in a local computer.

2.7 Ans.

Objects in the file system e.g. files, directories can be read/written/created/deleted using the rights of the local user who runs the program.

Network communication - the program might attempt to create sockets, connect to them, send messages etc.

Access to printers.

It may also impersonate the user in various ways, for example, sending/receiving email

2.8 Give some examples of applications where the use of mobile code is beneficial.

2.8 Ans.

Doing computation close to the user, as in Applets example

Enhancing browser- as described on page 70 e.g. to allow server initiated communication.

Cases where objects are sent to a process and the code is required to make them usable. (e.g. as in RMI in Chapter 5)

2.9 Consider a hypothetical car hire company and sketch out a three-tier solution to the provision of their underlying distributed car hire service. Use this to illustrate the benefits and drawbacks of a three-tier solution considering issues such as performance, scalability, dealing with failure and also maintaining the software over time.

2.9 Ans.

A three-tier solution might consist of:

- a web-based front-end offering a user interface for the car hire service (the presentation logic);
- a middle tier supporting the core operations associated with the car hire business including locating a particular make and model, checking availability and pricing, getting a quote and purchasing a particular car (the application logic);
- a database which stores all the persistent data associated with the stock (the data logic).

In terms of performance, this approach introduces extra latency in that requests must go from the web-based interface to the middle tier and then to the database (and back). However, processing load is also spread over three machines (especially over the middle tier and the database) and this may help with performance. For this latter reason, the three-tier solution may scale better. This may be enhanced though by other, complementary placement strategies including replication.

In terms of failure, there is an extra element involved and this increases the probability of a failure occurring in the system. Equally, failures are more difficult to deal with, for example if the middle tier is available and the database fails.

The three-tier approach is much better for evolution because of the intrinsic separation of concerns. For example, the middle tier only contains application logic and this should therefore be easier to update and maintain.

2.10 Provide a concrete example of the dilemma offered by Saltzer's end-to-end argument in the context of the provision of middleware support for distributed applications (you may want to focus on one aspect of providing dependable distributed systems, for example related to fault tolerance or security).

Saltzer's end-to-end argument states that communication-related functions can only be completely and reliably implemented with the knowledge and help of the application and therefore providing that function as a feature of the communication system itself (or middleware) is not always sensible. One concrete example is in secure communication. Assume that in a given system, the communication subsystem provides encrypted communication. This is helpful but insufficient. For example, the pathway from the network to the application software may be compromised and this is unprotected. This solution also does not deal with malicious participants in the exchange of data.

Consider also the reliable exchange of data implemented by introducing checksum protection on individual hops in the network. Again, this is insufficient as data may be corrupted by intermediary nodes, for example, gateways, or indeed in the end systems.

- 2.11 Consider a simple server that carries out client requests without accessing other servers. Explain why it is generally not possible to set a limit on the time taken by such a server to respond to a client request. What would need to be done to make the server able to execute requests within a bounded time? Is this a practical option?

2.11 Ans.

The rate of arrival of client requests is unpredictable.

If the server uses threads to execute the requests concurrently, it may not be able to allocate sufficient time to a particular request within any given time limit.

If the server queues the request and carries them out one at a time, they may wait in the queue for an unlimited amount of time.

To execute requests within bounded time, limit the number of clients to suit its capacity. To deal with more clients, use a server with more processors. After that, (or instead) replicate the service....

The solution may be costly and in some cases keeping the replicas consistent may take up useful processing cycles, reducing those available for executing requests.

- 2.12 For each of the factors that contribute to the time taken to transmit a message between two processes over a communication channel, state what measures would be needed to set a bound on its contribution to the total time. Why are these measures not provided in current general-purpose distributed systems?

2.12 Ans.

Time taken by OS communication services in the sending and receiving processes - these tasks would need to be guaranteed sufficient processor cycles.

Time taken to access network. The pair of communicating processes would need to be given guaranteed network capacity.

The time to transmit the data is a constant once the network has been accessed.

To provide the above guarantees we would need more resources and associated costs. The guarantees associated with accessing the network can for example be provided with ATM networks, but they are expensive for use as LANs.

To give guarantees for the processes is more complex. For example, for a server to guarantee to receive and send messages within a time limit would mean limiting the number of clients.

- 2.13 The Network Time Protocol service can be used to synchronize computer clocks. Explain why, even with this service, no guaranteed bound given for the difference between two clocks.

2.13 Ans.

Any client using the ntp service must communicate with it by means of messages passed over a communication channel. If a bound can be set on the time to transmit a message over a communication channel, then the difference between the client's clock and the value supplied by the ntp service would also be bounded. With unbounded message transmission time, clock differences are necessarily unbounded.

- 2.14 Consider two communication services for use in asynchronous distributed systems. In service A, messages may be lost, duplicated or delayed and checksums apply only to headers. In service B, messages may be lost, delayed or delivered too fast for the recipient to handle them, but those that are delivered arrive order and with the correct contents.

Describe the classes of failure exhibited by each service. Classify their failures according to their effect on the properties of validity and integrity. Can service B be described as a reliable

communication service?

2.14 Ans.

Service A can have:

arbitrary failures:

- as checksums do not apply to message bodies, message bodies can be corrupted.
- duplicated messages,

omission failures (lost messages).

Because the distributed system in which it is used is asynchronous, it cannot suffer from timing failures.

Validity - is denied by lost messages

Integrity - is denied by corrupted messages and duplicated messages.

Service B can have:

omission failures (lost messages, dropped messages).

Because the distributed system in which it is used is asynchronous, it cannot suffer from timing failures.

It passes the integrity test, but not the validity test, therefore it cannot be called reliable.

-
- 2.15 Consider a pair of processes X and Y that use the communication service B from Exercise 2.14 to communicate with one another. Suppose that X is a client and Y a server and that an invocation consists of a request message from X to Y (that carries out the request) followed by a reply message from Y to X. Describe the classes of failure that may be exhibited by an invocation.

2.15 Ans.

An invocation may suffer from the following failures:

- *crash failures*: X or Y may crash. Therefore an invocation may suffer from crash failures.
- *omission failures*: as SB suffers from omission failures the request or reply message may be lost.

-
- 2.16 Suppose that a basic disk read can sometimes read values that are different from those written. State the type of failure exhibited by a basic disk read. Suggest how this failure may be masked in order to produce a different benign form of failure. Now suggest how to mask the benign failure.

2.16 Ans.

The basic disk read exhibits arbitrary failures.

This can be masked by using a checksum on each disk block (making it unlikely that wrong values will go undetected) - when an incorrect value is detected, the read returns no value instead of a wrong value - an omission failure.

The omission failures can be masked by replicating each disk block on two independent disks. (Making omission failures unlikely).

-
- 2.17 Define the integrity property of reliable communication and list all the possible threats to integrity from users and from system components. What measures can be taken to ensure the integrity property in the face of each of these sources of threats

2.17 Ans.

Integrity - the message received is identical to the one sent and no messages are delivered twice.

threats from users:

- injecting spurious messages, replaying old messages, altering messages during transmission

threats from system components:

- messages may get corrupted en route
- messages may be duplicated by communication protocols that retransmit messages.

For threats from users - at the Chapter 2 stage they might just say use secure channels. If they have looked at Chapter 7 they may be able to suggest the use of authentication techniques and nonces.

For threats from system components. Checksums to detect corrupted messages - but then we get a validity problem (dropped message). Duplicated messages can be detected if sequence numbers are attached to messages.

- 2.18 Describe possible occurrences of each of the main types of security threat (threats to processes, threats to communication channels, denial of service) that might occur in the Internet.

2.18 Ans.

Threats to processes: without authentication of principals and servers, many threats exist. An enemy could access other user's files or mailboxes, or set up 'spoof' servers. E.g. a server could be set up to 'spoof' a bank's service and receive details of user's financial transactions.

Threats to communication channels: IP spoofing - sending requests to servers with a false source address, man-in-the-middle attacks.

Denial of service: flooding a publicly-available service with irrelevant messages.